

AI 활용 기술 문서 — 도플갱어 마을 탈출

팀/제작: BadWordFilter · 개발 기간: 2026-08-08 ~ 08-10 (사전 과제)

리포지토리: <https://github.com/BadWordFilter/doppelganger-village> (전 커밋에 AI 활용 내역 명시)

1. 요약 — "AI가 에디터를 직접 조작하는 개발 파이프라인"

이 프로젝트는 AI를 코드 생성기가 아니라 Unity 에디터를 직접 조작하는 개발 주체로 썼습니다. Claude (Fable 5, Claude Code)가 MCP(Model Context Protocol)로 Unity 에디터에 연결되어, 다음 사이클을 사람의 개입 없이 자율 수행했습니다:

C# 스크립트 작성 → 컴파일 에러 확인 → 씬/프리팹 구축(에디터 코드 실행)
→ 플레이 모드 진입 → Photon 실서버 접속 → UI 버튼 클릭·판정 시뮬레이션(리플렉션)
→ 상태 조회로 결과 검증 → 스크린샷 확인 → git 커밋

사람이 직접 한 일은 계정·자산 발급(Photon AppId, GitHub 리포·Pages), Asset Store 임포트 클릭, 푸시 버튼, 최종 플레이 감수뿐입니다. 이 문서와 게임 소개 문서 원고 역시 같은 세션에서 AI가 작성했습니다.

2. 사용 도구

도구	용도
Claude Code (Fable 5)	전체 개발 주도 — 코드, 씬 구축, 검증, 디버깅, 아트, 문서
MCP for Unity (CoplayDev, MIT)	Claude ↔ Unity 에디터 브리지 (스크립트/씬/컴포넌트/플레이 모드/콘솔/빌드)
Blender 5.2 (헤드리스)	AI가 작성한 파이썬 절차 모델링 스크립트로 캐릭터 11종(.glb) 생성 — 외부 모델 에셋 0
Python (pdfplumber, pypdf)	기획서 PDF에서 대화 테이블 200개 추출
Kiwi 형태소 분석기 (kiwipiepy)	PDF 추출 시 소실된 한국어 띄어쓰기를 줄바꿈 접합부에만 국소 복원

3. 핵심 활용 사례

3-1. 기획서 대화 테이블 → 게임 데이터 (원문 무손실 이식)

게임의 최대 자산인 대화 테이블 200개 전량(동물 9종 × 일상/핵심 질문·정상/이상 답변)을 기획서 PDF에서 **dialogue.json**으로 이식했습니다.

- pdfplumber 표 인식 → 페이지 경계에 잘린 10개 행은 원문 텍스트에서 복원
- PDF 추출 시 줄바꿈 지점에서 사라지는 띄어쓰기를 Kiwi 형태소 분석을 접합부에만 적용해 원문 훼손 없이 복원
- 검증 2중화: ① 전 항목을 공백 제거 후 PDF 원문과 부분 문자열 자동 대조 ② 전량 육안 검수 (수동 보정 2건)
- 임의 창작 0건 — 기획 원문이 데이터로 그대로 살아 있음
- 대사 속 울음소리·행동 지문은 텍스트로 두지 않고 키워드 감지 → 실제 합성 사운드·절차 애니메이션으로 공연 (목 720° 회전, 기어다니기, 폴짝 뛰기 등)

3-1b. 아트 파이프라인 — AI가 Blender를 코드로 조작

- 캐릭터 11종(동물 9종 + 고양이 플레이어 + 추격자)을 AI가 작성한 Blender 파이썬 스크립트로 절차 모델링 → 헤드리스 실행 → .glb 익스포트 → Unity 임포트
- 계층 규칙(Body/Head/EyeL·R)을 유지해 게임 내 연출 혹(머리 회전·눈 붕괴)과 호환
- sRGB→리니어 감마 보정, 종별 실루엣 차별화(양털 뭉치·강아지 늘어진 귀·늑대 갈가·박쥐 날개막 등)
- 카툰 아웃라인(뒤집힌 혈)·바닥 그림자·라운드 코너 9-슬라이스 UI 스프라이트까지 전부 코드 생성 — 외부 그래픽 에셋 0

3-1c. 사운드 파이프라인 — 절차 합성 오디오

- 효과음 12종 + 동물 울음 9종 + 낮/밤 앰비언트 루프를 전부 C# 코드로 파형 합성 (AudioClip.Create) — 외부 오디오 에셋 0
- 밤 공포 연출: 불협 저음 클러스터 + 심장박동, 추격자 근접 시 심장 소리 가속

3-2. 멀티플레이 아키텍처 구현 (마스터 권한 판정)

- 도플갱어 배정: 마스터가 룸 CustomProperties에 기록 → 늦게 합류한 클라이언트도 동일 결과
- 대화 확률 굴림(핵심 질문 차수별 10/30/60%)·판정·정산·승패를 전부 마스터 클라이언트가 굴리고 RPC로 브로드캐스트 — 전 클라이언트 동일 결과 보장
- 질문 횟수는 동물 1마리당 3회를 전 플레이어가 공유 — 협동 자원 관리가 되도록 설계

3-3. AI 자율 검증 (사람 없는 QA 루프)

Claude가 플레이 모드에서 Photon 실서버에 접속해 룸을 만들고, 리플렉션으로 대화 UI 버튼을 클릭하며 시나리오를 수행했습니다:

- 대화 왕복: 질문 클릭 → 마스터 굴림 → 답변 표시·카운트 갱신 확인
- 과잉 심문: 3회 소진 → 4번째 질문 → HP 100→66·동물 돌변 확인
- 엔딩 통합 테스트: 퇴치→도주→잠입→구출×4 → 정산 차감(구출 5-잠입 1=3) → "탈출 성공!" 화면까지 자동 검증

3-4. AI 디버깅 사례 (실제 트러블슈팅 로그)

문제	AI의 진단·해결
배포 후 게임 404	브라우저 네트워크 로그 분석 → .gitignore 의 Build/ 패턴이 docs/Build/ (배포 산출물)까지 무시함을 발견 → 루트 앵커링으로 수정
RPC 무반응	PUN 로그 "Illegal view ID:0" 추적 → 스크립트 기반 씬 저장 시 PUN 에디터 훅이 미작동 → sceneViewId 수동 할당
Player 타입 충돌	자체 네임스페이스와 Photon 타입 충돌 → using 별칭 적용
Unity 6 API 변경	Photon 구버전 코드의 API 경고 → API 업데이터 안내 및 검수

3-5. 대표 프롬프트 (요지)

- "기획서 PDF와 대화 요강을 분석해 실현 가능성을 판단하고 개발을 준비해줘" → 마감·제출 요건 리서치, 스코프 확정, 저장소·데이터 셋업
- "순서대로 다 만들어보자" → 4단계 구현·검증·커밋 사이클 자율 수행
- 세부 지시 없이도 CLAUDE.md(프로젝트 헌법)에 기획 규칙·아키텍처 규칙·구현 순서를 담아 AI가 항상 같은 기준으로 판단하게 운영

4. 외부 에셋 / 오픈소스 출처

자산	라이선스	용도
Photon PUN 2 / Photon Voice 2 (Exit Games)	Asset Store 무료 (Photon Cloud 무료 20 CCU)	멀티플레이 네트워킹
Pretendard 폰트 (길형진)	SIL OFL 1.1 (리포에 라이선스 동봉)	한글 UI 전체
MCP for Unity (CoplayDev)	MIT	AI-에디터 브리지 (개발 도구, 빌드 미포함)
Unity 6000.3.21f1 LTS / URP	Unity 라이선스	엔진
Blender 5.2	GPL (개발 도구, 빌드 미포함)	캐릭터 절차 모델링
glTFast (Unity 패키지)	Apache 2.0	.glb 임포트
pdfplumber·pypdf·kiwipie.py	MIT/Apache/LGPL (개발 도구, 빌드 미포함)	데이터 이식 파이프라인

그 외 모델·텍스처·사운드는 전부 AI가 코드로 생성 (외부 아트·오디오 에셋 0). 게임 내 대화 텍스트는 자체 기획서 원문.

5. 한계와 다음 단계

- LLM 실시간 NPC 대화는 의도적으로 배제 (지연·비용·멀티 동기화·API 키 노출 문제). 사전 과제는 데이터 드리븐 대화 + 확률 연출로 코어 재미를 증명하고, LLM 인게임 통합은 본선 과제로 설계
- 세션별 상세 로그: 리포 [submission/ai-log.md](#) (본 문서의 원천 데이터)